6 High-Level Design (Architecture)

Chapter 8. The rule now reverses: a high-level design that does not say where things run, what they are built from and what that choice cost has not been done.

6.1 Architectural drivers: quality-attribute scenarios

QAS	Quality attribute	Stimulus	Artifact	Response	Response measure	From
QAS-01	Performance	20 users simultaneously searching on normals conditions	Search subsystem	Returns matching game results	At least 95% of searches return within 3 seconds across 100 test searches	QR-01
QAS-02	Usability / Responsive design	A user opens a main page at 360 px or 1366 px viewport width	Web interface	Displays the page correctly	100% of tested main pages have no clipped or overlapping content at both widths	QR-02
QAS-03	Security	An unauthorized or unauthenticated user attempts a moderator-only action	Authorization / moderation subsystem	Denies access	100% of tested unauthorized attempts are blocked	QR-03

Architectural constraints. CON-01 and CON-02 shape the architecture by requiring the system to comply with the access restrictions of external platforms and by preventing unauthorized access to private or restricted third-party data. External game and review data must therefore be accessed only through approved interfaces and controlled components of the system.

6.2 Architectural style and alternatives

Style considered	Drivers it serves	What it costs	Verdict

Layered (3-Tier) with MVC-style separation	Supports QAS-01 performance, QAS-02 usability, and QAS-03 security by separating UI, application logic, authorization, and data access	Requires maintaining clear interfaces and multiple layers	Chosen
Microservices	Could separate search, reviews, moderation, and external data independently	Adds deployment, networking, monitoring, and coordination overhead that is unnecessary for the current scope	Rejected
Event-driven	Could support asynchronous updates and decouple some processing	Adds event infrastructure and complexity that the current use cases do not require	Rejected

The team selected a layered client-server architecture with MVC-style separation. This architecture supports QAS-01 by keeping search and data-access logic centralized on the server, supports QAS-02 by separating the presentation layer from application logic, and supports QAS-03 by keeping authorization decisions on the server rather than relying on the client. Microservices and event-driven architectures were rejected because their added deployment and coordination complexity would not provide enough benefit for the current project scope.

6.3 Component view

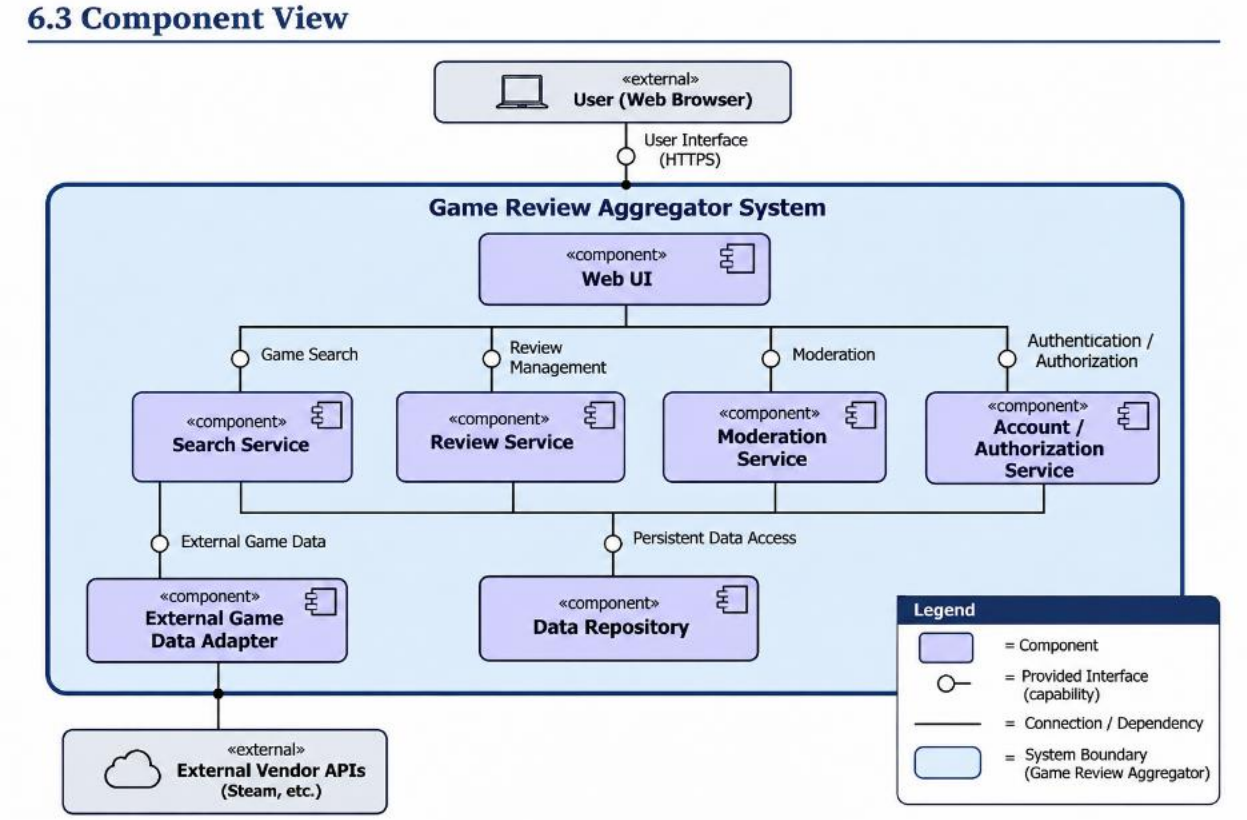


Figure 6.3 Component diagram: components, provided and required interfaces, external systems

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
Web UI	Presents search, game details, reviews, account, and moderation views to users	User Interface	Search Service, Review Service, Moderation Service, Account/Auth Service	UC-05, UC-06, UC-02, QAS-02
Search Service	Handles game search, filtering, and matching results	Game Search	Game Data Repository, External Game Data	UC-05, FR-01, FR-10, QAS-01
Review Service	Handles review creation, retrieval, sorting, and aggregate ratings	Review Management	Review Repository, Game Data Repository	UC-02, UC-06, FR-03–FR-07

Moderation Service	Handles reporting and moderator-only content actions	Moderation	Account/Auth Service, Review Repository	FR-08, FR-09, QAS-03
Account / Authorization Service	Handles user identity and permission checks	Authentication / Authorization	User Repository	QAS-03
Data Repository	Stores users, games, reviews, lists, and related application data	Persistent Data Access	Database	UC-02, UC-05, UC-06, QR-04
External Game Data Adapter	Retrieves permitted game/review information from third-party vendors	External Game Data	Vendor APIs	FR-01, FR-02, CON-01, CON-02

6.4 Deployment view

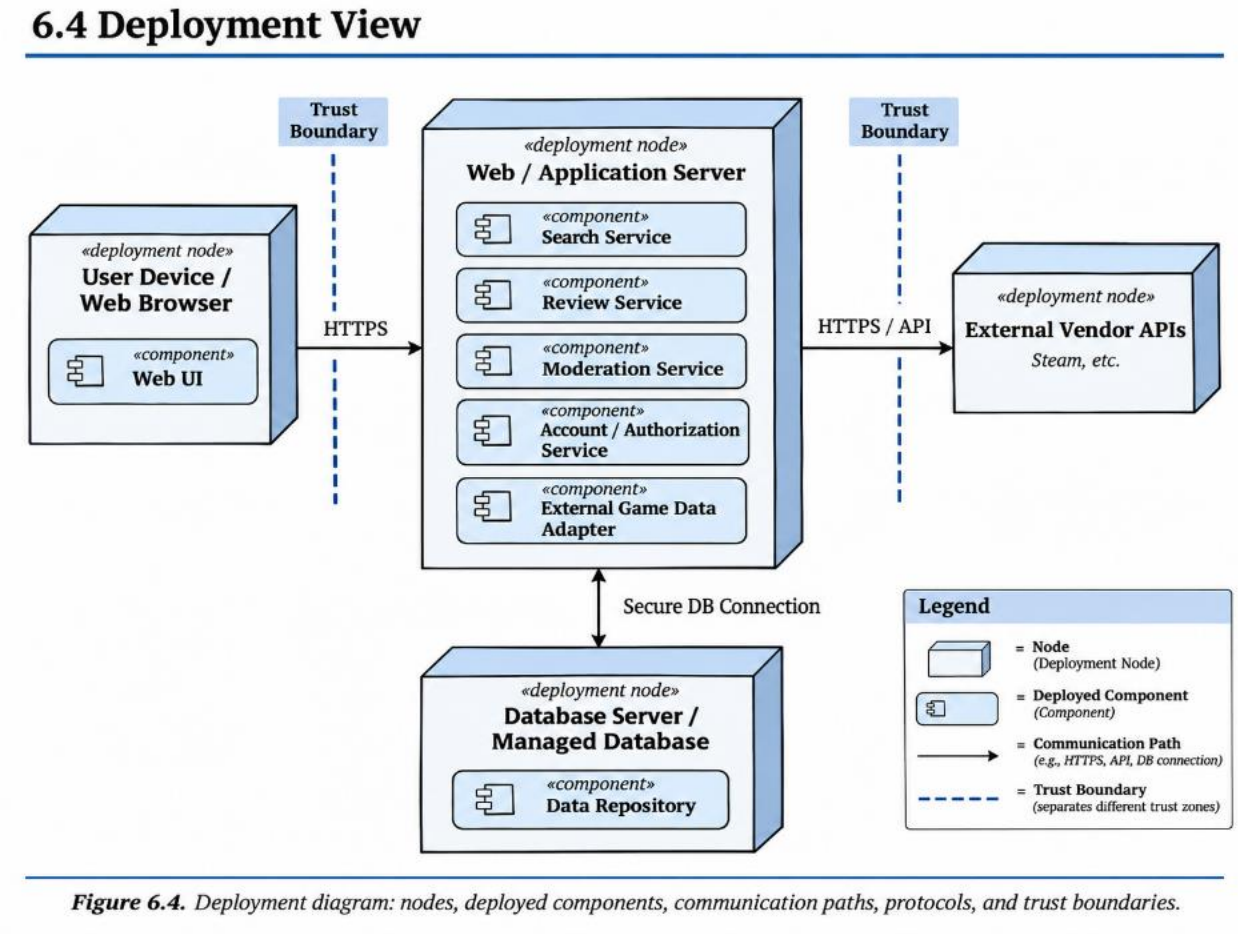


Figure 6.4 UML deployment diagram: nodes, execution environments, deployed artifacts, communication paths with protocols, trust boundaries

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
User Device / Web Browser	Web UI	Web/Application Server via HTTPS	User-controlled / untrusted
Web / Application Server	Search Service, Review Service, Moderation Service, Account/Auth Service, External Game Data Adapter	Browser via HTTPS; Database via database connection; Vendor APIs via HTTPS	Team-controlled application zone
Database Server / Managed Database	Data Repository	Web/Application Server via database protocol/secure connection	Team-controlled data zone
External Vendor APIs	Steam and other permitted external game/review data sources	External Game Data Adapter via HTTPS/API	External / third-party trust zone

6.5 From analysis to architecture

Analysis object (from 5.1 and 5.2)	Component that owns it (6.3)	Note (split, merged, external)
Gamer	Account / Authorization Service	User/account identity is managed here
Game	Search Service	Main domain object used for searching and displaying game information
GameReview	Review Service	Review creation, retrieval, sorting, and ratings belong here
Filter	Search Service	Used to refine game searches
List	Review Service	User-created game lists are treated as user content
SearchController	Search Service	Interaction/control object merged into the Search Service
SearchQuery	Search Service	Search interaction object used to

		represent search criteria
SearchResult	Search Service	Search interaction object used to represent matching results

6.6 Design principles applied

Principle	Where it shows in your design	Evidence (component, interface, ADR)
Separation of concerns	Search, reviews, moderation, account/authorization, UI, and external-data access are handled by separate components instead of one large component	Component view 6.3
Information hiding	Vendor-specific API details are hidden behind the External Game Data Adapter, so other components do not need to know how each vendor API works	External Game Data Adapter
Low coupling	Web UI communicates with application services rather than directly accessing the database or vendor APIs	Web UI interfaces / Component view 6.3
High cohesion	Search Service contains search and filtering responsibilities, while Review Service contains review and rating responsibilities	Search Service and Review Service
Design for change	External vendor access is isolated in the External Game Data Adapter, so a provider can be changed or added without redesigning the rest of the system	External Game Data Adapter
Keep it simple (KISS)	The design uses a layered client-server structure rather than microservices or event-driven architecture because the current project scope does not require the added complexity	Section 6.2 / ADR-01